C# OOP Cheat Sheet - Midterm

Made By Oni
github.com/VladOniscenko

This cheat sheet is a study aid based on the sample exams. It does not guarantee what will be on the actual midterm. Always check the official course materials!

1. Properties with Validation

What: Use auto-properties or backing fields to control data access.

```
private int _age;
public int Age {
    get => _age;
    set => _age = (value < 0) ? 0 : value;
}
// Auto-property (simpler, no validation):
public string Name { get; set; }
```

Why: Backing field (_age) lets you validate in setter before storing.
Common exam Q: "How do you prevent invalid age?" → Use property setter with validation.
Mistake: Forgetting backing field when you need validation logic.

2. Inheritance & Constructors

What: Child class reuses parent code. Constructor must call base().

```
public class Person
{
    public string Name { get; set; }
    public int Age{ get; set; }
}

public class Member : Person {
    public Member(string name, int age)
        : base(name, age) {
        // Pass to parent
    }
}
```

Why: : base() ensures parent fields are initialized first.
Common exam Q: "What happens if you forget : base()?" → Parent constructor NOT called, fields stay null.
Mistake: Forgetting : base() or passing wrong number of args.

3. Interfaces & Lists

What: Interface = contract. List<Interface> holds ANY object that implements it.

```
public interface IFly { void Fly(); }
public class Bird : IFly {
    public void Fly()
    { Console.WriteLine("Bird flying!"); }
}

List<IFly> flyers = new List<IFly>();
flyers.Add(new Bird());
foreach (IFly f in flyers)
{
    f.Fly(); // Works for ANY object that implements IFly
}
```

Why: List<IFly> is polymorphic—holds Bird, Plane, Drone, anything that can Fly().
Common exam Q: "Why use List<IFly> instead of List<Bird>?" → Interface = flexible, can add other flying types.
Mistake: Forgetting to implement interface method in the class.

4. Type Checking (is/as)

What: Safe way to check if object is a specific type before casting.

```
// Option 1: is (returns bool)
if (book is EBook) { Console.WriteLine("Digital copy!"); }
// Option 2: as (returns object or null)
EBook ebook = item as EBook;
if (ebook != null) { Console.WriteLine(ebook.Download()); }
// Bad: Direct cast (throws exception if wrong type!)
EBook bad = (EBook)item; // ❌ Crash if item isn't EBook
```

Why: is/as are safe. Direct cast throws InvalidCastException if type doesn't match.
Common exam Q: "When to use is vs as?" → is = just check. as = check AND use immediately.
Mistake: Using direct cast without checking first.

5. CompareTo for Sorting

What: Implement IComparable to define sort order. Return negative/0/positive.

```
public class Animal : IComparable<Animal> {
    public string Name { get; set; }
    public int Age { get; set; }
    public int CompareTo(Animal other) { // Sort by Name first
        int nameSort = this.Name.CompareTo(other.Name); if (nameSort != 0)
            return nameSort;
        // If Names equal, sort by Age
        return this.Age.CompareTo(other.Age);
    }
}
```

Why: Return negative = this first, 0 = equal, positive = other first.
Common exam Q: "Sort by last name, then first name?" → Check last name first in CompareTo.
Mistake: Not tiebreaking (only sorting by one field).

6. Polymorphism (virtual/override)

What: Child class can override parent method with custom logic.

```
public class Person {
    public virtual void PrintInfo()
    { Console.WriteLine("I am a person"); }
}
public class Member : Person
{
    public override void PrintInfo()
    {
        base.PrintInfo();
        // Call parent first
        Console.WriteLine("And I'm a member!");
    }
}
```

Why: virtual = parent allows override. override = child changes behavior but can still call base.
Common exam Q: "Make Bird.Fly() different from Animal.Fly()?" → Use override.
Mistake: Forgetting **virtual** in parent = override won't compile.

7. Generic Constraints

What: Limit what types can be used with a generic method/class.

```
// T must inherit Animal AND implement IPet
public void Add<T>(T item) where T : Animal, IPet {
    // Can safely call Animal methods AND IPet methods
    Console.WriteLine(item.GetAge());
    item.Adopt();
}
// T must have parameterless constructor
public T Create<T>()
where T : new()
{
    return new T();
}
// T must be a class (reference type)
public void Process<T>(T item)
where T : class { }
```

Why: Constraints prevent compile errors. Compiler ensures T has required methods.
Common exam Q: "Why use where T : IComparable?" → So compiler knows T has CompareTo().
Mistake: Forgetting constraints, then calling methods that don't exist on T.

8. IEquatable<T> & Equals()

What: Define when two objects are "equal". Needed for .Contains(), .IndexOf() in lists.

```
public class Animal : IEquatable<Animal>
{
    public string Name { get; set; }
    public int Age { get; set; }
    public bool Equals(Animal other) {
        if (other == null)
            return false;
        return (this.Name == other.Name && this.Age == other.Age);
    }
    public override bool Equals(object obj) {
        if (obj is Animal animal)
            return Equals(animal);
        return false;
    }
}
```

Why: Default Equals only checks if same reference. You need to define "same data".
Common exam Q: "Does list.Contains(newAnimal) find a duplicate?" → Implement IEquatable.
Mistake: Only implementing IEquatable method, forgetting override object.Equals().

9. Constructor Overloading

What: Multiple constructors with different parameters in same class.

```
public class Book {
    public string Title { get; set;}
    public string Language { get; set; }
    // Constructor 1: Title only
    public Book(string title) {
        Title = title;
        Language = "English";
    }
    // Constructor 2: Title + Language
    public Book(string title, string language) {
        Title = title;
        Language = language;
    }
}
```

Why: Flexibility—create Book with just title, or with title+language. Same class, different ways.
Common exam Q: "Allow creating Book two ways?" → Use constructor overloading.
Mistake: Two constructors with same parameter types (compiler won't distinguish them).

Notes